

Problem Domain 1 Suppose a scenario that School of Computer and Systems Sciences (SC&SS) requires a new student registration system. The School is working with much older system developed around mainframe technology. As the time passes, the older system facing some maintenance issues and lack of some advance features. The School decided to replace it with a new client-server system-based Student Registration Process. The new system will allow students to register for courses and view report cards from personal computers attached to the campus LAN. Professors will be able to access the system to sign up to teach courses as well as record grades. As the University is facing some funding issues and not able to support the School significantly, the School cannot afford to replace the entire system at one go. The School will keep the old existing course catalog database where all course information is maintained. This database is an Ingres relational database running on a DEC VAX. Fortunately, the School has invested in an open SQL interface that allows access to this database from School's UNIX servers. The legacy system performance is rather poor, so the new system must ensure that access to the data on the legacy system occurs in a timely manner. The new system will access course information from the legacy database but will not update it. The registrar's office will continue to maintain course information through another system. At the beginning of each semester, students may request a course catalogue containing a list of course offerings for the semester. Information about each course, such as professor, department, and prerequisites, will be included to help students make informed decisions. The new system will allow students to select four course offerings for the coming semester. In addition, each student will indicate two alternative choices in case the student cannot be assigned to a primary selection. Course offerings will have a maximum of thirty students and a minimum of three students. A course offering with fewer than three students will be canceled. For each semester, there is a period of time that students can change their schedule. Students must be able to access the system during this time to add or drop courses. Once the registration process is completed for a student, the registration system sends information to the billing system so the student can be billed for the semester. If a course fills up during the actual registration process, the student must be notified of the change before submitting the schedule for processing. At the end of the semester, the student will be able to access the system to view an electronic report card. Since student grades are sensitive information, the system must employ extra security measures to prevent unauthorized access. Professors must be able to access the on-line system to indicate which courses they will be teaching. They will also need to see which students signed up for their course offerings. In addition, the professors will be able to record the grades for the students in each class for his/her own subject. The system will also generate a final report card. TO HELP TO INDIAN AUTHOUR BOOK.

ANS:-----

Alright, let's break down the needs and steps for implementing this new student registration system while ensuring it aligns with the budget constraints and existing infrastructure.

Requirements:

1. System Components:

- **Student Registration:** Allows students to register for courses, view schedules, and report cards.
- **Professor Interface:** Allows professors to manage course sign-ups, view student lists, and input grades.
- **Course Catalogue:** Accesses the existing Ingres database for course information.
- **Billing Integration:** Sends registration details to the billing system.

2. Key Features:

- **Client-Server Architecture:** Ensures efficient performance and scalability.
- **Course Selection:** Limits of 30 students per course, with a minimum of 3.
- **Alternative Course Selection:** Allows students to choose alternate courses.
- **Schedule Change Period:** Allows for add/drop of courses within a specific period.
- **Secure Access:** Ensures data security, especially for grades.

Implementation Plan:

Phase 1: Analysis and Design

- **Needs Analysis:** Detailed requirement gathering from all stakeholders (students, professors, administration).
- **System Design:** Architecture planning including database schema, server setup, user interfaces.

Phase 2: Development and Testing

- **Development:**
 - Build the client-server architecture.
 - Develop the student registration and professor interfaces.
 - Integrate the open SQL interface with the legacy database.
 - Ensure secure authentication and authorization mechanisms.
- **Testing:**
 - Unit Testing: Individual components.
 - Integration Testing: Ensure new system components interact seamlessly.
 - User Acceptance Testing: Involving a group of students and professors to test real-world scenarios.

Phase 3: Deployment and Transition

- **Pilot Deployment:** Roll out the system in phases to ensure smooth transition and minimize disruption.
- **Training:** Provide training sessions for students and professors.
- **Support and Maintenance:** Establish a support team for troubleshooting and future maintenance.

Considerations:

- **Budget Constraints:** Focus on incremental development and deployment to spread out costs.
- **Performance:** Optimize SQL queries and ensure efficient data access from the legacy system.
- **Security:** Implement strong security measures like encryption and multi-factor authentication.

Example for Indian Professors:

Since this might be designed with Indian academia in mind, here's a nod to localized needs:

- **Localization:** Support for multiple Indian languages if needed.
- **Cultural Fit:** Tailored training sessions considering cultural contexts.

What do you think of this approach? Does it align with the needs and constraints you've mentioned? Let's make sure we cover everything!

Problem Domain 2 You have been tasked to develop the software for a mine pump control system, designed to monitor and pump flood water out of mine shafts. As underground mining operations take place far below the water table, flooding into mine galleries and shafts is an ever-present danger. Excessive flooding is clearly a safety hazard for workers, but also has profitability implications ranging from equipment damage to productivity delays, to mine closures in extreme circumstances. The system to be developed will be required to monitor the water level in a given mine shaft using two sensors. A high water sensor that measures the maximum acceptable level of flooding in a shaft before pumping begins, and a low water sensor, which measures the minimum level of acceptable flooding and pumping stops. These sensors are used to start a mine pump. When the flooding level exceeds the level determined by the high water sensor the pump is switches on. When the water has been pumped out and the minimum level of acceptable flooding has been reached, as measured by the low water sensor, the pump switches off. In addition to flooding mining is often hindered by methane pockets, where gas seeps into the shafts and galleries triggering an evacuation. Again this is a safety hazard, the mining staff won't be able to breathe, and even more critically, operating equipment may generate sparks which will cause the methane to ignite. Therefore, the system will include a methane sensor that will be used to trigger an evacuation alarm in the presence of dangerous levels of methane (measured in N parts per million), and also switch off the pump regardless of the current water level. The system is used by two key roles. The first is the Operator. This role is required to log in to the system with a username and password. Following a successful login, the operator is able to start or stop the pump if, and only if, the water level is between the high and low sensor limits. The second role is the Supervisor. A supervisor must verify their security credential as per the operator above. Following Supervisor. A supervisor must verify their security credential as per the operator above. Following a successful login, they are able to switch the pump on, or off at any time. For example, a supervisor could run the pump after the flood level has dropped below the level set by the low water sensor. They could also switch the pump off if the water level goes over the maximum high level of flooding. In these cases, the supervisors' actions override the automatic behavior of the pump. A supervisor is required to "reset" the pump system in order to re-establish automatic behavior. Finally, to meet Federal monitoring standards a persistent log is required to capture the following events: Pump switched on by high water sensor Pump switched off by low water sensor Pump switched on or off by operator or supervisor Evacuation alarm triggered by methane sensor The reading of the methane sensor every 30 minutes The reading of the methane sensor (for the last 24 hours) can be read by the operator. All readings (up to 30 days) can be read by the supervisor. The supervisor also has the capability to add a "note" to any specific log event that occurs within 24 hour.

Ans:---

Problem Domain Analysis: Mine Pump Control System

Objective:

To design and develop a **Mine Pump Control System** that ensures the safety and efficiency of mining operations by monitoring flooding and methane levels. The system must:

- Automate the operation of a mine pump based on **flood water levels**.
 - Trigger an **evacuation alarm** in the presence of dangerous **methane gas** levels.
 - Provide manual override capabilities for **Operators** and **Supervisors**.
 - Maintain a **persistent event log** to comply with federal monitoring standards.
-

Functional Requirements

1. Automatic Pump Operation

- **High Water Sensor:**
 - Start the pump when water exceeds the high water sensor level.
 - Log the event: "Pump switched on by high water sensor."
- **Low Water Sensor:**
 - Stop the pump when water is reduced to the low water sensor level.
 - Log the event: "Pump switched off by low water sensor."

2. Methane Gas Monitoring

- **Methane Sensor:**
 - Trigger an **evacuation alarm** if methane levels exceed a critical threshold NNN parts per million.
 - Automatically stop the pump regardless of water levels.
 - Log the event: "Evacuation alarm triggered by methane sensor."

3. User Roles and Access

- **Operator:**
 - Must log in with a username and password.
 - Can manually start or stop the pump only if the water level is between the high and low sensor limits.
 - Can view methane readings for the **last 24 hours**.
- **Supervisor:**
 - Must log in with a username and password.
 - Can manually start or stop the pump **at any time**, overriding automatic behavior.
 - Can reset the pump system to restore automatic behavior.
 - Can view all methane readings (up to **30 days**).
 - Can add notes to log events within the past **24 hours**.

4. Event Logging

- Persistently log the following:
 - Automatic pump actions by sensors (start/stop).
 - Manual pump actions by Operator or Supervisor.
 - Methane sensor readings (every 30 minutes).
 - Methane sensor-triggered evacuation alarms.
 - Allow log retrieval:
 - **Operators:** Last 24 hours.
 - **Supervisors:** Last 30 days.
 - Allow Supervisors to add notes to specific log events (within 24 hours).
-

Non-Functional Requirements

1. **Reliability:**
 - Ensure that sensor data is accurately processed and pump operations occur without delay.
 - System must operate continuously in a potentially hazardous mining environment.
 2. **Security:**
 - Protect user credentials and ensure role-based access to features.
 - Secure log data from unauthorized access or modification.
 3. **Scalability:**
 - Support additional sensors or more pumps in future system upgrades.
 4. **Usability:**
 - Provide a user-friendly interface for Operators and Supervisors.
 - Display real-time sensor readings and system status clearly.
 5. **Compliance:**
 - Adhere to federal safety and monitoring regulations.
-

Key System Modules

Module	Description
Sensor Interface	Collects data from high water, low water, and methane sensors.
Pump Controller	Automatically or manually controls the pump based on sensor readings and user actions.
User Management	Manages login, role-based access control, and user credentials.
Event Logger	Captures and stores events, including sensor-triggered actions, manual pump overrides, methane readings, and alarm triggers.
Data Viewer	Provides Operators and Supervisors access to logs and real-time sensor readings as per their role.
Supervisor Tools	Allows Supervisors to override pump behavior, reset the system, and add notes to log entries.

System Workflow

Automatic Pump Behavior:

1. **Flood Monitoring:**
 - If water exceeds the high water level, the system starts the pump and logs the event.
 - If water drops below the low water level, the system stops the pump and logs the event.
2. **Methane Monitoring:**
 - If methane levels exceed NNN, the system:
 - Stops the pump.
 - Triggers an evacuation alarm.
 - Logs the alarm event.

Manual Pump Control:

- **Operator:**
 - Logs in and checks water level.
 - Starts/stops the pump (if water level is between sensor limits).
 - Logs actions: "Pump switched [on/off] by operator."
- **Supervisor:**
 - Logs in and overrides pump behavior (anytime).
 - Resets the system to restore automatic control.
 - Logs actions: "Pump switched [on/off] by supervisor."

Event Logging and Notes:

- **Operators:**
 - View methane readings (last 24 hours).
 - **Supervisors:**
 - View methane readings (last 30 days).
 - Add notes to log events (within 24 hours).
-

Proposed Architecture

1. **Layered Architecture:**
 - **Sensor Layer:**
 - Interfaces with water and methane sensors for real-time data collection.
 - **Application Layer:**
 - Implements business logic for pump control, user roles, and event logging.
 - **Data Layer:**
 - Stores persistent logs and user credentials in a secure database.
2. **Technology Stack:**
 - **Backend:**
 - Python (Django or Flask) for core logic.
 - **Frontend:**

- HTML5, CSS, and JavaScript (React or Vue.js) for user interface.
 - **Database:**
 - PostgreSQL or MySQL for log storage.
 - **Hardware:**
 - Microcontroller interface for sensor data collection and pump control.
-

Use Cases

1. Sensor-Based Automatic Pump Operation:

- **Actors:** High water sensor, Low water sensor.
- **Description:** Sensors control pump actions and log events.

2. Methane Alarm and Safety:

- **Actors:** Methane sensor.
- **Description:** Triggers evacuation alarm and stops the pump.

3. Manual Override:

- **Actors:** Operator, Supervisor.
- **Description:**
 - Operator starts/stops the pump (limited by water levels).
 - Supervisor starts/stops the pump anytime and can reset the system.

4. Log Management:

- **Actors:** Operator, Supervisor.
 - **Description:**
 - View logs (24 hours for Operators, 30 days for Supervisors).
 - Supervisors add notes to logs within 24 hours.
-

Expected Outcomes

1. Improved safety by automating pump control and monitoring methane levels.
2. Enhanced manual control capabilities for emergency scenarios.
3. Compliance with federal monitoring standards through robust event logging.
4. Secure and efficient system supporting real-time data processing.